

program_guide

プログラム解説ガイド

0. 全体の目的：何を解きたいか

【現実の問題】

工作機械のスピンドル（主軸）が回転すると熱が発生し、
機械本体が少しずつ伸び縮みする。
この「熱変位」が加工誤差の原因になる。

【難しい点】

機械全体の温度分布を知りたいが、
センサをたくさん取り付けることはコスト・スペースの制約で難しい。
→ センサは数点しか使えない。

【このプログラムが答える問い】

「2 点しかセンサがなくても、FEM モデルとカルマンフィルタを組み合わせれば
センサのない場所の温度と熱変位を高精度に推定できるか？」

この問いを、1 次元熱棒（10 節点）という小さなモデルで実証するのが
このプログラム群の目的である。

1. ファイル構成と役割分担

```
001_kalman_thermal_1d/
|
|—— thermal_model.py    ← 「物理モデル」を作る
|                        （熱伝導方程式 → 状態空間モデル → 離散化）
|
|—— kalman_filter.py    ← 「推定アルゴリズム」
|                        （カルマンフィルタの予測・更新を実行）
|
|—— main.py             ← 「メインシナリオ：温度センサ 2 点のみ」
|                        シミュレーション実行 + 8 パネルグラフ生成
|
```

└── main_disp_obs.py ← 「拡張シナリオ：変位センサを追加」
3 ケース比較（温度のみ / 温度 + 変位 / 変位のみ）

各ファイルは独立したモジュールであり、main.py と main_disp_obs.py が
thermal_model.py と kalman_filter.py を import して使う。

2. thermal_model.py：物理モデルを作る

2-1. 何をモデル化しているか

熱源 Q [W/m]
↓（節点 0 に集中）
[0] — [1] — [2] — [3] — [4] — [5] — [6] — [7] — [8] — [9]
|対流|対流
T_amb=20°CT_amb=20°C

- 素材：鋼材相当（熱拡散率 $\alpha = 1.2 \times 10^{-5} \text{ m}^2/\text{s}$ ）
- 棒の長さ：0.9 m（節点間距離 $dx = 0.1 \text{ m}$ ）
- 境界条件：両端が空気対流冷却（ $h_{\text{conv}} = 8 \text{ W/m}^2\text{K}$ ）

10 個の節点それぞれに温度状態がある。状態ベクトルは：

$$\theta = [\theta_0, \theta_1, \dots, \theta_9]^T \in \mathbb{R}^{10}$$

2-2. 連続時間の方程式から離散時間へ（_build_continuous → _discretize）

ステップ①：連続系の状態方程式を立てる（_build_continuous）

熱拡散方程式を有限差分で離散化すると：

$$\dot{\theta} = A_c \theta + B_c u$$

A_c は三重対角行列で、各要素の物理的意味：

```
r = alpha / dx**2    # 熱拡散による隣接節点への熱流れ [1/s]
h = h_conv / (rho_cp * dx) # 対流冷却の強さ [1/s]
```

```
# 内部節点 i（左右の熱流れの差）
A[i, i-1] = +r    # 左隣から受け取る熱
A[i, i]   = -2r   # 自分から左右に流れ出す熱
A[i, i+1] = +r    # 右隣から受け取る熱
```

```
# 端部節点（対流冷却が加わる）
A[0, 0] = -r - h  # 左方向の熱流れがなく、対流冷却あり
A[0, 1] = +r
```

$\mathbf{B}_c$ は節点 0 にだけ入力が入るベクトル：

$B[0, 0] = 1.0 / (\text{rho_cp} * \text{dx})$ # 熱入力 Q [W/m] を温度変化率 [$^{\circ}\text{C/s}$] に変換

ステップ②：前進オイラー法による離散化 (`_discretize`)

カルマンフィルタはデジタルコンピュータで動くため、

連続時間方程式を「タイムステップ $\text{dt}=10\text{s}$ ごとの差分方程式」に変換する必要がある。

微分 $\dot{\theta}$ を前進差分で近似する（前進オイラー法）：

$$\frac{\theta_{k+1} - \theta_k}{\Delta t} \approx \mathbf{A}_c \theta_k + \mathbf{B}_c u_k$$

整理すると離散化行列が得られる：

$$\mathbf{A}_d = \mathbf{I} + \mathbf{A}_c \Delta t, \quad \mathbf{B}_d = \mathbf{B}_c \Delta t$$

コードでは：

```
A_d = np.eye(self.n) + self.A_c * self.dt    # I + A_c · Δ t
B_d = self.B_c * self.dt                    # B_c · Δ t
```

離散化後の状態方程式（カルマンフィルタが使う形）：

$$\theta_{k+1} = \mathbf{A}_d \theta_k + \mathbf{B}_d u_k + \mathbf{w}_k$$

2-3. step メソッド：真値の生成に使う

```
def step(self, x, u, noise_std=0.0):
    x_next = A_d @ x + B_d @ u          # モデルによる時間発展
    x_next += randn(n) * noise_std      # プロセスノイズを加える
    return x_next
```

これは「シミュレーション上の正解（真値）」を作るためだけに使う。

カルマンフィルタ自身はこのメソッドを使わない

(KF は自分の内部で同じ $\mathbf{A}_d$, $\mathbf{B}_d$ を使って予測する)。

3. kalman_filter.py：推定アルゴリズム

3-1. カルマンフィルタとは何か

カルマンフィルタは「FEM モデルの予測」と「センサの計測」を

最適な重みで組み合わせて、最も確からしい状態推定値を求めるアルゴリズム。

【予測】

前のステップの推定値 $\hat{x}_{k-1}$
→ A_d を使って 1 ステップ進める
→ 「モデルだけによる予測」 $\hat{x}_k^-$ を得る

【更新】

センサの計測値 y_k (ノイズあり) が届く
→ 「予測との差 (イノベーション)」を計算
→ カルマンゲイン K で「どれだけセンサを信頼するか」を重み付け
→ $\hat{x}_k = \hat{x}_k^- + K (y_k - H \hat{x}_k^-)$ で補正

3-2. コードの全体像

```
class KalmanFilter:
    def __init__(self, A, B, H, Q, R, x0, P0):
        self.x = x0    # 状態推定値 (10 節点の温度)
        self.P = P0    # 誤差共分散行列 (推定がどれくらい信頼できるか)
```

4 つの行列の役割：

行列	形状	意味
$A (= A_d)$	10×10	状態遷移 (FEM モデル由来)
$B (= B_d)$	10×1	入力行列 (熱源の位置と強さ)
H	$p \times 10$	観測行列 (どの節点をセンサで見るか)
Q	10×10	プロセスノイズ (モデル誤差の大きさ)
R	$p \times p$	観測ノイズ (センサ精度)

3-3. predict メソッド

```
def predict(self, u):
    self.x = self.A @ self.x + self.B @ u    # モデルで時間発展
    self.P = self.A @ self.P @ self.A.T + self.Q    # 不確かさも伝播させる
```

P は誤差共分散行列。「この推定値はどれくらい信頼できるか」を表す。

時間が経つほど予測の不確かさは増えるので、 $+Q$ で不確かさを加算する。

3-4. update メソッド

```
def update(self, y):
    S = self.H @ self.P @ self.H.T + self.R    # イノベーション共分散
    K = self.P @ self.H.T @ inv(S)              # カルマンゲイン
    innov = y - self.H @ self.x                 # イノベーション (予測との差)
```

```

self.x = self.x + K @ innov          # センサ情報で補正
self.P = (I - K @ self.H) @ self.P  # 不確かさを減らす

```

カルマンゲイン K の直感的意味：

$$K = \frac{\text{モデルの不確かさ}}{\text{モデルの不確かさ} + \text{センサの不確かさ}}$$

- P が大きい（モデルが信頼できない）→ K が大きい → センサを強く信頼
- R が大きい（センサがノイズだらけ）→ K が小さい → モデルを信頼

3-5. uncertainty プロパティ

```

@property
def uncertainty(self):
    return np.sqrt(np.diag(self.P)) # 各節点の推定標準偏差  $\sigma$ 

```

P の対角成分の平方根が「その節点の推定誤差の標準偏差」。

これが小さいほど推定が信頼できる。

4. main.py：メインシナリオの実行

4-1. このファイルが行うこと（大きな流れ）

1. 設定パラメータを定義
2. 熱入力パターンを生成（スピンドルの ON/OFF サイクル）
3. シミュレーションループ（90 分間 = 540 ステップ）：
 - a. 真値を 1 ステップ進める（thermal_model.step）
 - b. センサノイズを加えて「計測値 y 」を作る
 - c. カルマンフィルタで予測・更新
 - d. 推定値と不確かさを記録
4. 熱変位を計算（全節点温度の合計から）
5. 結果をグラフに描画 → results.png に保存

4-2. 観測行列 H の組み立て

```

SENSOR_NODES = [0, 9] # 節点 0 と節点 9 にセンサを配置

```

```

H = np.zeros((2, 10)) # 2 行 10 列
H[0, 0] = 1.0          # 1 行目：節点 0 の温度を観測
H[1, 9] = 1.0          # 2 行目：節点 9 の温度を観測

```

行列 H はカルマンフィルタに「どの節点が計測されているか」を教える。

H を変えるだけでセンサ配置を変えられる。

4-3. シミュレーションループ（核心部分）

```
for k in range(N_STEPS):
    u = np.array([u_series[k]])          # 現在の熱入力

    # 真値の更新（シミュレーション上の正解）
    x_true = model.step(x_true, u, noise_std=PROC_NOISE_STD)

    # センサ計測（真値 + ガウスノイズ）
    y = H @ x_true + randn(n_obs) * OBS_NOISE_STD

    # カルマンフィルタ：予測 → 更新
    x_est, P_est = kf.step(u, y)

    # 記録
    true_T[k] = x_true
    kf_T[k]    = x_est
    kf_sigma[k] = sqrt(diag(P_est))
```

重要なのは「真値」と「KF の推定値」が別々に管理されている点。

`x_true` は物理シミュレーションの正解。`x_est` がカルマンフィルタの推定。

4-4. 熱変位の計算

```
def compute_displacement(temps, t_ref=20.0):
    delta_T = temps - t_ref          # 各節点の温度上昇
    return ALPHA_EXP * sum(delta_T) * DX * 1e6  # [ $\mu$ m]
```

数式： $\delta = \alpha_{\text{exp}} \sum_{i=0}^9 (\theta_i - \theta_{\text{ref}}) \cdot \Delta x \cdot 10^6$

全節点の温度上昇を積算するので、

推定が不正確な節点があると変位の誤差も大きくなる。

（だからこそセンサのない節点を正確に推定することが重要）

4-5. グラフの 8 パネル構成

パネル	何を示すか
(1) 真の温度場	「問題設定」の確認。10 節点の真値と計測値（ノイズあり）を表示
(2) 未観測節点の温度推定	KF の主成果。真値（破線）vs 推定値（実線）、 $\pm 2\sigma$ の不確かさ帯
(3) 推定誤差の時系列	過渡域→定常域の収束過程。定常域の RMSE を凡例に表示

(4) 熱変位の比較	棒全体の伸び：真値 vs KF 推定。ピーク値と推定精度を表示
(5) 熱入力パターン	スピンドルの ON/OFF サイクル (10min ON / 5min OFF)
(6) 温度場スナップショット	4 つの時刻での空間分布 (位置 vs 温度)
(7) 推定不確かさの収束	σ (推定標準偏差) が時間とともに小さくなる様子
(8) 結果サマリテキスト	全節点の RMSE と変位 RMSE を一覧表示

5. main_disp_obs.py：変位センサを観測に加えた比較実験

5-1. このファイルが答える問い

「変位センサ（リニアエンコーダ等）の計測値を
カルマンフィルタの観測ベクトルに直接組み込むことで
推定精度は向上するか？」

3 つのケースを比較：

ケース	観測内容	観測行列 H の形状
Case A	温度センサ 節点 0・9	2 行 × 10 列
Case B	温度センサ 節点 0・9 + 変位センサ	3 行 × 10 列
Case C	変位センサのみ	1 行 × 10 列

5-2. 変位センサを H に組み込む方法（重要）

変位は全節点温度の線形和：

$$\delta[\mu\text{m}] = \alpha_{\text{exp}} \cdot \Delta x \cdot 10^6 \cdot \sum_{i=0}^9 (\theta_i - T_{\text{ref}})$$

これを整理すると：

$$\delta[\mu\text{m}] + C_{\text{disp}} \cdot n \cdot T_{\text{ref}} = C_{\text{disp}} \cdot \sum_{i=0}^9 \theta_i$$

$$\underbrace{y_{\text{disp,eff}}}_{\text{観測値として渡す}} = \underbrace{C_{\text{disp}} \cdot [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]}_{\mathbf{H}_{\text{disp}}} \cdot \boldsymbol{\theta}$$

ここで $C_{\text{disp}} = \alpha_{\text{exp}} \cdot \Delta x \cdot 10^6 = 12 \times 10^{-6} \times 0.1 \times 10^6 = 0.12 \mu\text{m}/^\circ\text{C}$

コードでは：

```
C_DISP = ALPHA_EXP * DX * 1e6    # = 0.12 μm/°C/node

H_disp = C_DISP * np.ones((1, N_NODES))    # [0.12, 0.12, ..., 0.12] (10 成分)

# 観測値を作るとき (真値 + ノイズ)
disp_true_um = C_DISP * sum(x_true - T_INIT)
y_disp_eff = disp_true_um + C_DISP*N_NODES*T_INIT + noise
```

$y_{\text{disp_eff}}$ (有効変位観測値) は「バイアスを除去した変位」であり、

$H_{\text{disp}} @ \theta$ と一致するように設計されている。

5-3. Case B での観測行列の組み立て

```
# H (3×10 行列)
H_B = np.vstack([H_temp, H_disp])
# = [[1, 0, 0, 0, 0, 0, 0, 0, 0, 0], ← 節点 0 の温度
#    [0, 0, 0, 0, 0, 0, 0, 0, 0, 1], ← 節点 9 の温度
#    [0.12, ..., 0.12]] ← 変位 (全節点の加重和)

# R (3×3 行列)
R_B = block_diag([[σ_temp^2, 0],
                  [0, σ_temp^2]],
                  [[σ_disp^2]])
```

5-4. 3 ケースの比較ループ

```
# 真値は全ケース共通 (公平な比較のため)
x_true_all = generate_true_values(model, u_series)

for case in ["A", "B", "C"]:
    np.random.seed(42)    # 観測ノイズのシードも揃える
    cases[case] = run_case(case, x_true_all, u_series)
```

真値を共通にして観測ノイズのシードも揃えることで、

ケース間の差は「観測行列 H の違いのみ」によるものになる。

5-5. グラフの 6 パネル構成

パネル	何を示すか
(1) 節点 1 の温度推定	未観測節点 1 を 3 ケースで比較
(2) 節点 2 の温度推定	未観測節点 2 を 3 ケースで比較

(3) 推定誤差（節点 1）	各ケースの誤差時系列 + RMSE
(4) 熱変位の比較	3 ケースの変位推定精度を比較
(5) 推定不確かさ σ	節点 2 の σ 収束速度の違い
(6) サマリ棒グラフ	未観測節点平均 RMSE（棒）+ 変位 RMSE（折線）

6. データの流れ（全体図）

【thermal_model.py】

ThermalBarModel

```

|—— _build_continuous() → A_c, B_c（連続系行列）
|—— _discretize()        → A_d, B_d（離散化行列）← カルマンフィルタに渡す
|—— step(x, u)           → x_next（真値の時間発展）
    ↓ 真値 x_true
    ↓

```

【main.py / main_disp_obs.py】

```

|—— 観測行列 H を設定（センサ配置に応じて）
|—— センサ計測 y = H @ x_true + ノイズ
    ↓ (A_d, B_d, H, Q, R)
    ↓ (初期値 x0, P0)
|   【kalman_filter.py】
|   KalmanFilter
|   |—— predict(u)      →  $\hat{x}^-$ ,  $P^-$ 
|   |—— update(y)       →  $\hat{x}$ , P（推定値と不確かさ）
|       ↓  $\hat{x}$ （全 10 節点の推定温度）
|—— compute_displacement( $\hat{x}$ ) → 熱変位推定値 [ $\mu$  m]

```

7. 設定パラメータの意味と変更効果

パラメータ	デフォルト値	意味	変更すると
N_NODES	10	節点数	増やすと解像度向上・計算量増
DT	10 s	タイムステップ	小さくすると精度向上・計算量増
N_STEPS	540	総ステップ（=90min）	シミュレーション時間
SENSOR_NODES	[0, 9]	センサ位置	[0, 4, 9] で 3 センサに変更可
Q_ON	6500 W/m	スピンドル熱入力	大きくすると温度上昇・変位増

PROC_NOISE_STD	0.03 °C	プロセスノイズ	大きくするとセンサを重視
OBS_NOISE_STD	0.20 °C	センサノイズ	センサの精度を模擬
DISP_NOISE_STD	0.50 μ m	変位センサノイズ	小さくすると Case B/C が改善

8. 試してみると面白い実験

実験 1: センサ配置を変える

SENSOR_NODES = [0, 4, 9] # 中央付近にも 1 点追加

→ 未観測節点の RMSE がどれくらい減るか確認

実験 2: センサを減らす (1 点のみ)

SENSOR_NODES = [0] # 片端のみ

→ 観測可能性 (Observability) の限界を確認

実験 3: ノイズ比を変える

PROC_NOISE_STD = 0.30 # モデル誤差を大きく設定 → センサを信頼
OBS_NOISE_STD = 0.02 # 高精度センサ → モデルより計測を重視

→ カルマンゲイン K の値がどう変わるか確認

実験 4: 変位センサの精度を上げる (main_disp_obs.py)

DISP_NOISE_STD = 0.10 # 高精度リニアエンコーダを想定

→ Case B と Case C の精度がどれだけ向上するか確認